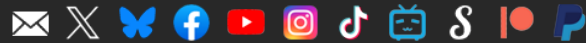
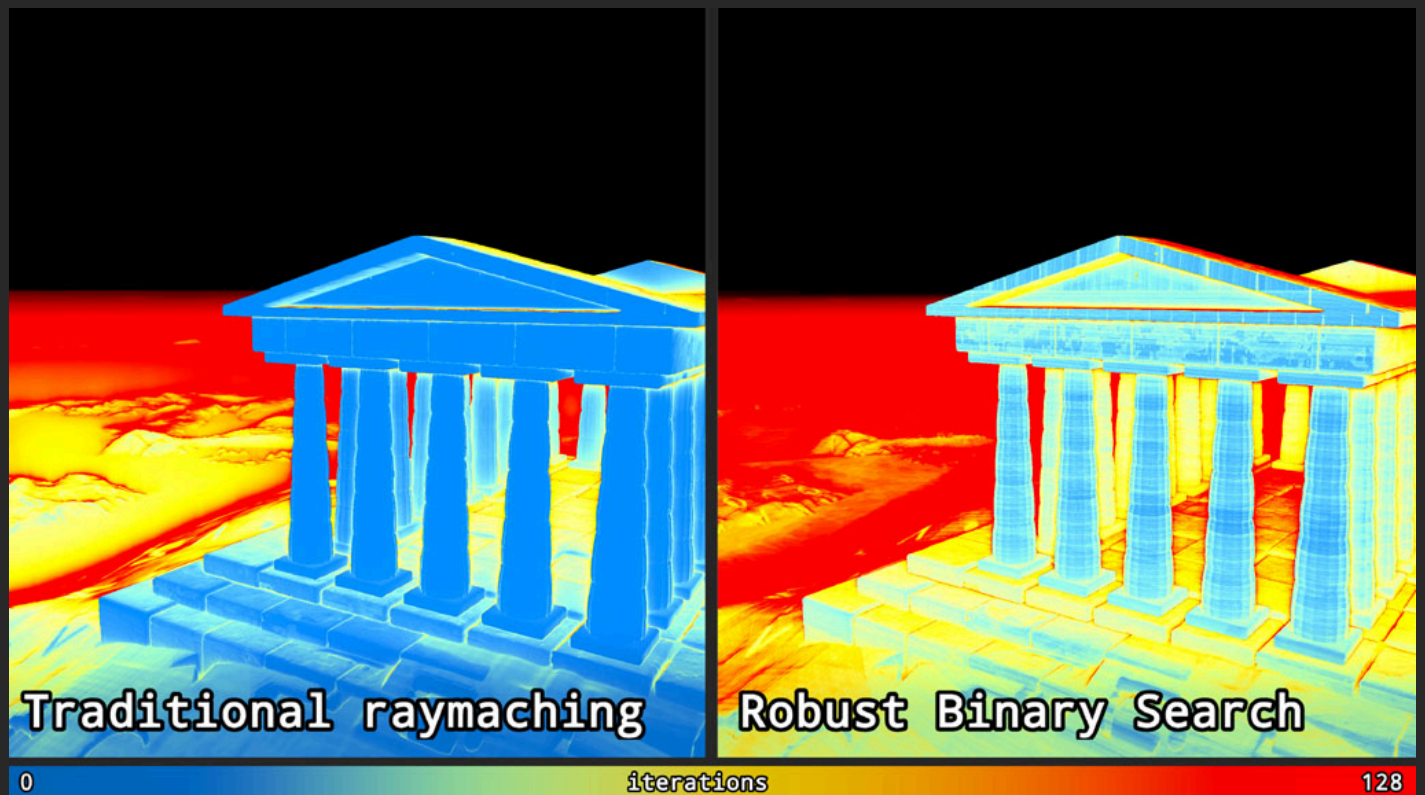


Inigo Quilez :: articles :: binary search for sdfs - 2018, 2022



This is an experiment I made in 2018 around the same week I was [playing with BVH and KDTree traversal for distance queries](#) in an attempt to accelerate SDF raycasting. While BVH partitions the dataset and KDTree partitions 3D space, the technique in this article partitions the ray itself



Left, traditional SDF raymarching. Right, binary search (uses more iterations/SDF evaluations)

The idea was to explore some raycasting method that would improve on the regular raymarching technique we usually employ for SDFs, by which I mean the uncomplicated distance based marching:

```
float raycast( vec3 ro, vec3 rd, float tmin, float tmax )
{
    float t = tmin;
    for( int i=0; i<kMaxIterations && t<tmax; i++ )
    {
        float d = map( ro + t*rd );
        if( d<t*kPixel ) return t;
    }
}
```

```
    t += d;  
}  
return -1.0;  
}
```

Here t_{min} is the the start value of the raymarch, often 0.0 camera rays, and t_{max} which is the furthest distance in the scene that we want to explore for intersections. So generally the length of the ray is $t_{len} = t_{max} - t_{min}$.

First attempt

Now, my ray partitioning idea was a binary search for intersections, by using the fact that evaluating the SDFs at any point in space provides us with valuable information about large regions of empty space where we are guaranteed not to encounter any geometry, which can be very useful. Imagine for example that we probe our SDF right in the middle of the ray at $t_{mid} = (t_{max} + t_{min})/2$, and see how near is the closest surface in the scene. If this distance is larger than $t_{len}/2$, ie, the half of the length of our segment, then we know our ray cannot intersect the SDF.

If that's the case we do not need to raymarch the ray at all. However, chances are this early exit strategy won't work often in practice, for the ray segment is really large and so the volume we need to be empty around it is really large.

No problem, let's split the ray segment right in the middle at t_{mid} into two sub-segments, a "front" segment going from t_{min} to t_{mid} , and a "back" segment going from t_{mid} to t_{max} . These new segments are half the length, and so they do have a greater chance of not overlapping any of the scene's geometry. So, we can now proceed like we did for the original ray's segment, one subsegment at a time. Let's start with the front segment and evaluate its center at $t_{mid1} = (t_{mid} + t_{min})/2$ and its half-size or radius as $t_{rad0} = (t_{mid} - t_{min})/2$. We can now evaluate the SDF at t_{mid1} , see what we get. If the SDF reports the closest distance is larger than t_{rad0} , we know the first half of our original segment does not intersect any geometry. So we can forever discard that first half, and only do our search for intersections on the second half. That's great, we shaved off half our raymarching efforts!

Now, whether that was the case and we now need to deal with the back segment or the front segment wasn't overlap free, determining where to search for intersections can be resolved by subdividing either of the segments in half yet again. And in fact, we can keep doing this recursively for as long as we need until we reach a recursion level where the length of the sub segments are short enough that it's fair to consider a

potential overlap between them and the SDF a point of intersection.

The implementation was pretty straightforward, but was based on a stack:

```
float raycast( vec3 ro, vec3 rd, float tmin, float tmax)
{
    stack_reset();
    vec2 s = vec2( tmin, tmax );           // start with the root

    for( int i=0; i<kMaxIterations; i++ )
    {
        float tcen = 0.5*(s.y + s.x);
        float trad = 0.5*(s.y - s.x);
        float d = map( ro + tcen*rd );     // intersect segment with SDF

        if( d < trad )                     // segment overlaps geometry
        {
            if( trad<tcen*kPixel )         // stop recursion
            {
                return s.x;
            }
            else                             // subdivide segment
            {
                stack_push( vec2( tcen, s.y ) ); // push back seg on the stack
                s = vec2( s.x, tcen);         // and start traversing the front
            }
        }
        else                               // pop next node from the stack
        {
            if( stack_is_empty() ) break;
            s = stack_pop();
        }
    }

    return -1.0;
}
```

Here each stack entry stores the two end points of the segment, so floating point values, but in the actual implementation I got it down to a single uint32.

The raycaster actually worked well, in that it was doing what it was supposed to do. But the rendering performance of my scene had dropped to about half of what I was getting with the original naive SDF raymarching loop.

As Shadertoy user Nicolas Klenert reported a year later, once can improve the performance of this technique by modifying the segment split to exclude the empty space reported by d . This needs a bit of precision control, which I added on top of his improvement, to finally look like this (changes marked in yellow):

```
...
if( d < trad )
{
    float th = tcen*kPixel;
    if( trad<th )
    {
        return s.x;
    }
    else
    {
        stack_push( vec2( tcen+d-th, s.y ) );
        s = vec2( s.x, tcen-d+th);
    }
}
else
{
    ...
}
```

This modification brings about 1.3x speed up, but unfortunately it's still not sufficient to surpass or even be competitive with the naive rendering method. Also, while a local optimization, it deviates us from the natural progression into the solution outlined next in this article.

Regardless, this was a weekday night experiment and I couldn't afford investigating further, but given the stack was likely taking a large number of registers and I was asking the code to index into it, something that wasn't recommended back in 2018, I assumed the performance problem was due to the stack anyways.

Second attempt

As showcased with Nicolas' example above, Shadertoy is a gift to me, I keep getting inspired by other artists and programmers. Another example is four years after the initial experiment, I saw user "fad" implement **the same binary search idea** but not applied to SDFs, but to general implicits. Because general implicits don't provide a measure of distance (which is the magic ingredient that makes SDFs so convenient for rendering) they were using Interval Arithmetic to bound the extent of the field, ie, the maximum possible volume the implicit function is taking, effectively providing a way to do the segment-implicit overlap test. From there on,

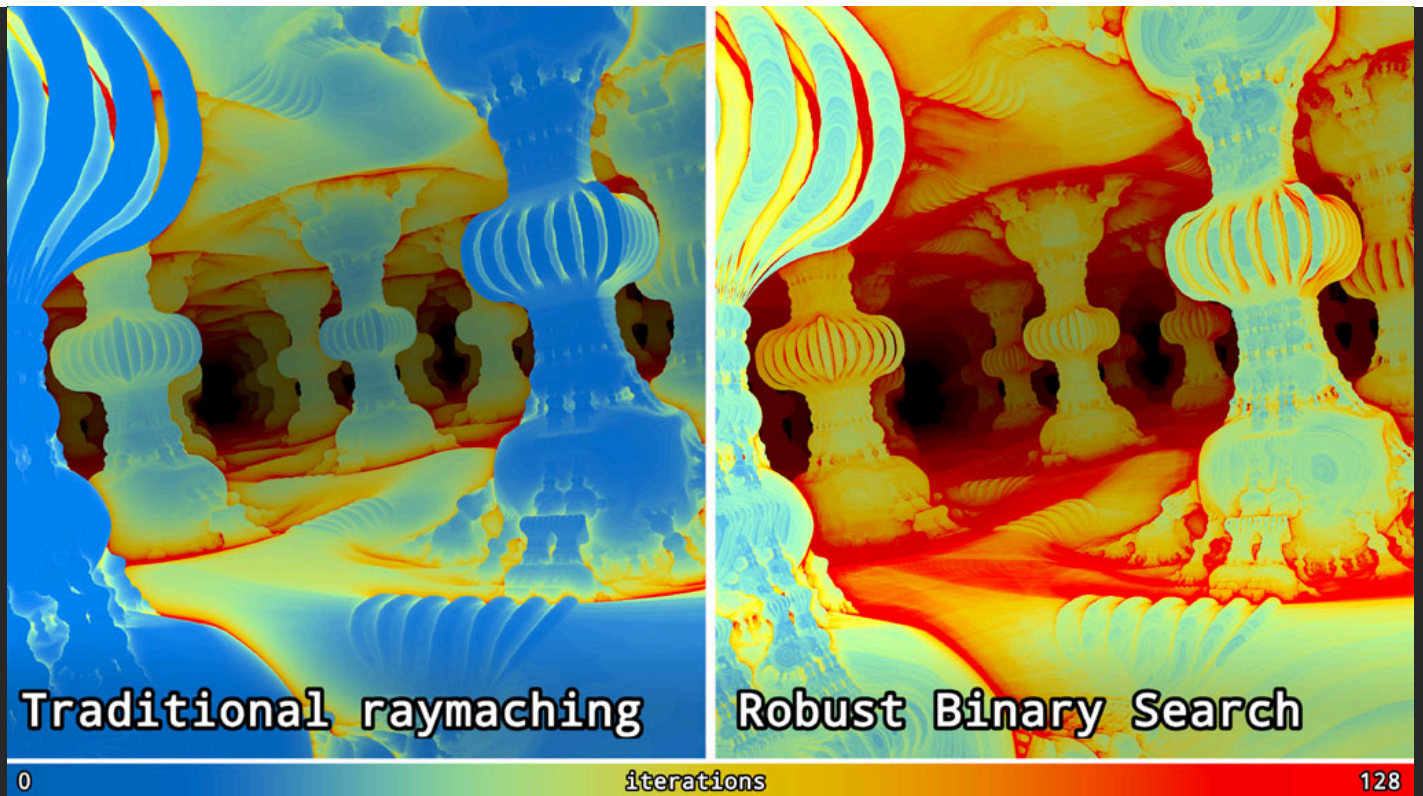
the method was like mine, except for one detail. There was no stack in the shader.

Of course everything clicked quickly - I was aware of the clever quadtree address systems that allows one to navigate them without pointers and stacks. There's for example Archee's shader from 2013 doing so, (see here <https://www.shadertoy.com/view/sIVfzz> and also here <https://www.shadertoy.com/view/MldXWX>) and the literature is full of such examples from when people started porting raytracers to the GPU in the mid 2000s. But somehow it didn't click for me until now. Not that I spent more than a night thinking of this problem, but still, it's funny how sometimes a thought that you shelf in a corner of your brain really sinks under all the other thoughts and never comes back to your "working set". Until somebody publishes a shader in Shadertoy. Yet another reason I love the site.

Anyway, I quickly went to my shader from 2018 and replaced the stack with the address base binary tree navigation. The shader worked again without problems, and now that I no longer had a stack I was excited to see that binary search kicked the ass of the distance based raymarcher.

Only that it didn't. The performance was still about half of what it should have been in order to be competitive. The stack hadn't been the problem. Or if it had been in 2018, it no longer was for sure in 2022.

It's only when I visualized the number of SDF evaluations per ray that everything became clear: the binary search was invoking the SDF way too many times. I did set the ray segment length threshold to match the epsilon tolerance in the naive SDF raymarcher to make sure both were resolving geometry to the same accuracy so the comparison was fair. And yes, definitely the binary search was too computation heavy. Here you can see the comparison between both methods.



Not the binary search method needs a lot more iterations to converge

So the conclusion is clear. As with many other attempts to be smart about raymarching SDFs with different techniques, it is not trivial to beat the embarrassingly uncomplicated SDF raymarcher loop (in single, non-packet/frustum/cone raycasting mode)

The stackless traversal

Now, because this article is a bit of a boomer, let me quickly add some quick explanations on the stackless recursion so we end the article with some useful information for those who might find the method useful on its own. This is how it works:

The traversal order is depth-first, and always visits the front segments before the back ones, so any intersection found is always the closest one to the camera. The nodes of the tree represent the subdivided segments, which the full tmin to tmax segment store at the root, its front and back subsegments stored in its two children nodes, etc. So, for a 4 levels recursion this would be the tree:

level	segment
-------	---------

0	1							
1	2				3			
2	4		5		6		7	
3	8	9	10	11	12	13	14	15

The traversal is done in depth-first order, which means that assuming we never skipped subtrees and all segments of the tree were visited, they would be visited in this order:

1 2 4 8 9 5 10 11 3 6 12 13 7 14 15 = END

While this sequence feels a bit arbitrary at first, traveling across nodes of the tree is actually very systemating and done with simple manipulation of the node number, like this:

In order to recurse down to a left children we go to $\text{node} = \text{node} * 2$

In order to skip the subtree under us, we repeat $\text{node} = \text{node} / 2$ (integer division) climbing up the tree until the result is even, at which point, we do $\text{node} = \text{node} + 1$ to move to the right sibling.

You can see the code here below, and also in the following live Shadertoy demo:

<https://www.shadertoy.com/view/wlGcWG>.

```
uint BasePowerOfTwo(uint x) // [256,511] -> 256
{
    #if 0
        x |= x >> 16;
        x |= x >> 8;
        x |= x >> 4;
        x |= x >> 2;
        x |= x >> 1;
        return x^(x>>1);
    #else
        return 1u<<(uint(floor(log2(float(x)))));
    #endif
}

float raycast( vec3 ro, vec3 rd, float tmin, float tmax )
{
    uint nod = 1u; // start at the root, ie, the
                  // whole [tmin,tmax] segment
    const int kMaxLevel = 18; // recursion (worst case will
    while( true ) // be 2^kMaxLevel iterations
```

```

{
    uint bas = BasePowerOfTwo(nod);
    uint seg = nod - bas;
    float tle = (tmax-tmin)/float(bas);           // compute center and radius
    float tce = tmin+tle*(float(seg)+0.5);        // of current node
    float tra = 0.5*tle;                          // in "t" ray space

    float d = map( ro + tce*rd );                  // evaluate SDF at segment center

    if( d<tra )                                    // now, if SDF overlaps the segment
    {
        if( tra<tce*kPixel || lev>kMaxLevel )     // and if segment is tiny
        {
            return tce-tra;                      // consider this an intersection
        }
        else                                     // otherwise, continue subdivision
        {
            nod <=<= 1; // by going to left child
        }
    }
    else                                         // but if they don't overlap
    {                                           // then skip the subtree by
        for( ; (nod&1u)==1u; nod>>=1 );        // climbing to ancestor
        if( nod==0u ) break;                  // and if not root, then
        nod++;                                // moving to right sibling
    }
}

return -1.0;
}

```

inigo quilez - learning computer graphics since 1994